Signals

- Event -Driven Programming
  - Events
  - Event handlers
  - Example from Java swing: event listener, event handler

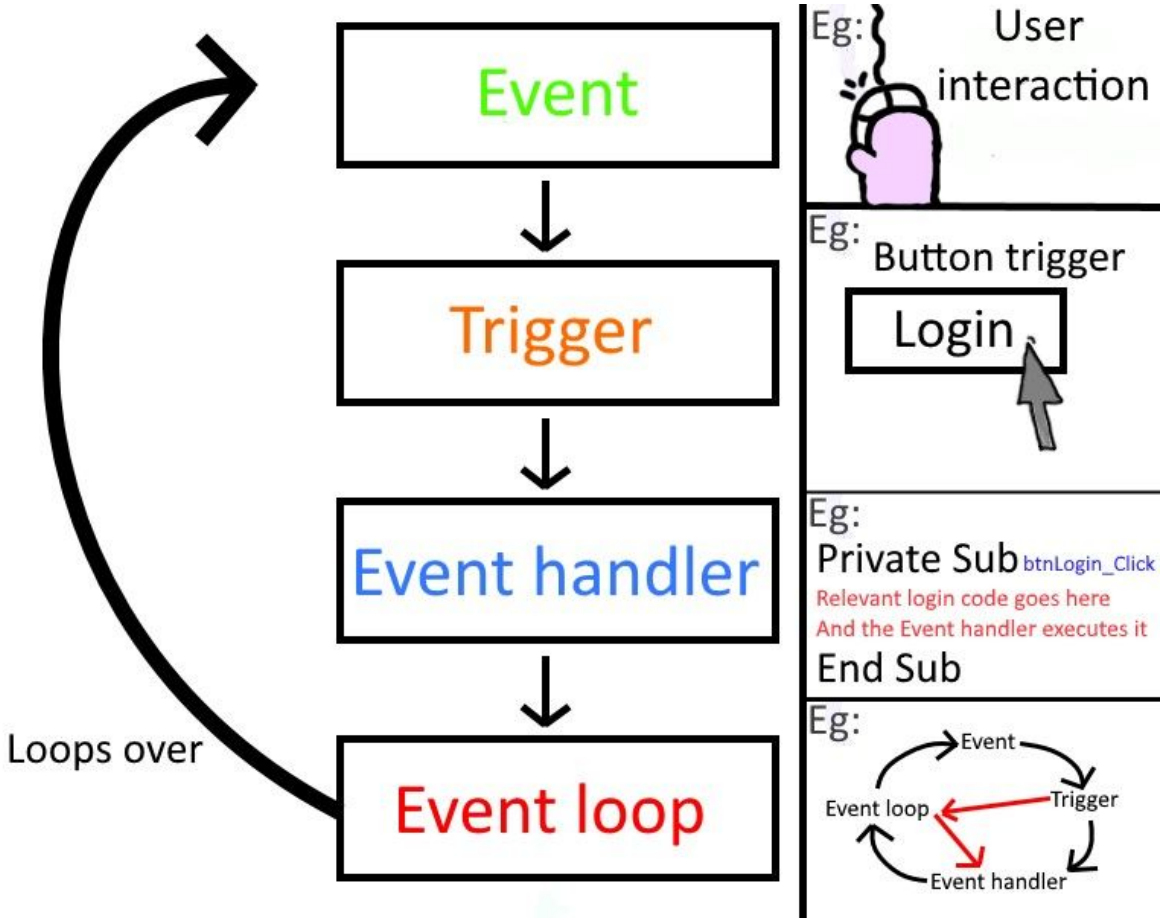
- Signal programming
  - Signals
  - Signal handlers

The content for signals are copied from
https://github.com/angrave/SystemProgramming/wiki

Event Driven Programming

Imperative or Procedural programming: code is executed by procedural order

EDP: Code is executed upon the activation of events

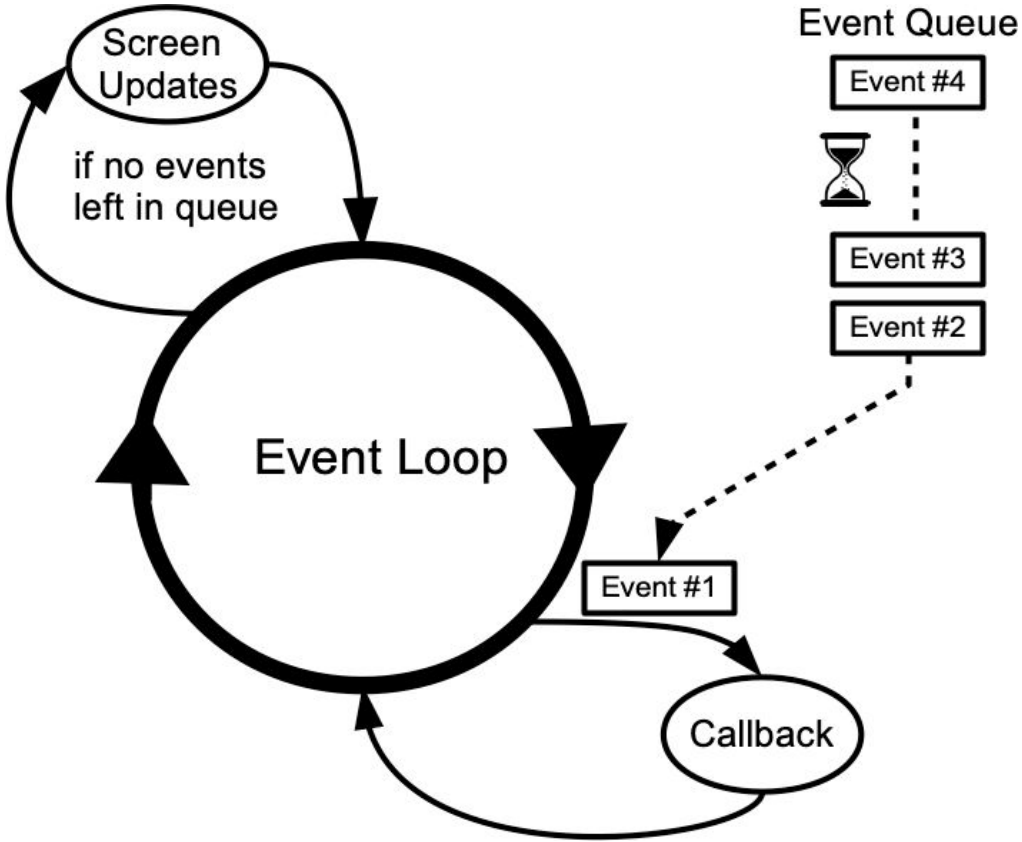


https://textbooks.cs.ksu.edu/cc410/ii-gui/13-event-driven-programming/index.html

Java swing example

Bind an event to a function: Event handler (in java terminology listener)

Event loop: Runs within a GUI thread and listens for events generated by a user



https://textbooks.cs.ksu.edu/cc410/ii-gui/13-event-driven-programming/index.html

Example event handlers

```java
import java.awt.event.*;
import javax.swing.*;
class MyEventHandler1 implements ActionListener{
   @Override
   //This is event handler-1
   public void actionPerformed(ActionEvent e){
       JButton b = (JButton)e.getSource();
       b.setText("A Click Event");
   }
}
```

```java
import java.awt.event.*;
class MyEventHandler2 implements ActionListener{

   @Override
   //This is event handler-2
   public void actionPerformed(ActionEvent e){
       System.out.println("This is second handler for click event: ");
   }
}
```

Both handlers are called when click-event occurs on button

```
import javax.swing.*;
class EventGenerator {
   public static void main(String[] args){
       JFrame frame = new JFrame();
       JButton button = new JButton("Hi");

       MyEventHandler1 handler1 = new MyEventHandler1();
       MyEventHandler2 handler2 = new MyEventHandler2();

       button.addActionListener(handler1);
       button.addActionListener(handler2);

       frame.add(button);
       frame.setSize( 150, 100 );
       frame.setVisible(true);
   }
}
```

Signals are very similar to EDP

- Standardized messages sent to a running process
- Limited form of IPC

a bit of terminology.

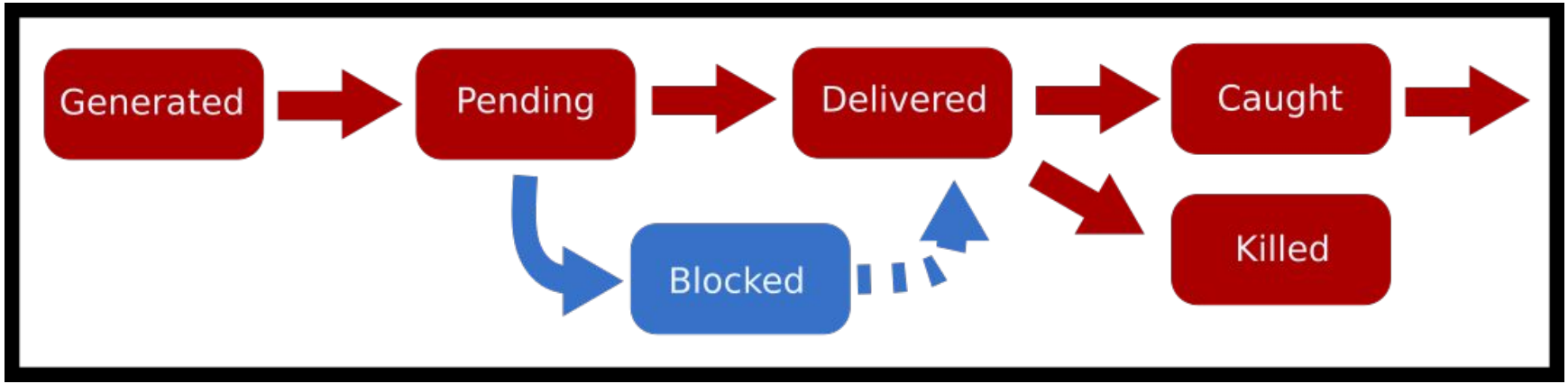
A signal disposition is a per-process attribute that determines how a signal is handled after it is **delivered**.

- Think of it as a table of signal-action pairs.

The full discussion is in the http://man7.org/linux/man-pages/man7/signal.7.html. The actions are

1. `TERM`, terminates the process
2. `IGN`, ignore
3. `CORE`, generate a core dump
4. `STOP`, stops a process
5. `CONT`, continues a process
6. Execute a custom function

Flow chart



Signals can be generated in multiple ways.

The system can send an event.

- For example, if a process accesses a page that it isn't supposed to, the hardware generates an interrupt which gets intercepted by the kernel. The kernel finds the process that caused this and sends a software interrupt signal `SIGSEGV`.
- There are other kernel events like a child being created or a process needs to be resumed.

Signals can be generated in multiple ways.

The user can send a signal.

- For example, you are at the terminal, and you press `CTRL-C`.
- One can also use the built-in `kill` to send any signal.

```c
#include <unistd.h>
#include <stdio.h>
int main() {
  printf("My pid is %d\n", getpid() );
  int i = 60;
  while(--i) {
    write(1, ".",1);
    sleep(1);
  }
  write(1, "Done!",5);
  return 0;
}
```

```
$ ./program &
My pid is 403
...
$ kill -SIGSTOP 403
$ kill -SIGCONT 403
...
```

In c program

```
kill(child, SIGUSR1); // Send a user-defined signal
kill(child, SIGSTOP); // Stop the child process (the child cannot prevent this)
kill(child, SIGTERM); // Terminate the child process (the child can prevent this)
kill(child, SIGINT); // The equivalent to CTRL-C (by default closes the process)
```

Another command `killall` works the exact same way but instead of looking up by PID, it tries to match the name of the process. `ps` is an important utility that can help you find the pid of a process.

```
# First let's use ps and grep to find the process we want to send a signal
to
$ ps au | grep myprogram
angrave  4409   0.0  0.0  2434892    512 s004  R+    2:42PM   0:00.00
myprogram 1 2 3

#Send SIGINT signal to process 4409 (The equivalent of `CTRL-C`)
$ kill -SIGINT 4409

# Send SIGKILL (terminate the process)
$ kill -SIGKILL 4409
$ kill -9 4409
# Use kill all instead to kill a process by executable name
$ killall -l firefox
```

To send a signal to the running process, use `raise` or `kill` with `getpid()`.

```
raise(int sig); // Send a signal to myself!
kill(getpid(), int sig); // Same as above
```

Summary

Processes can send messages to each by using signals.

- **This should be only used in low-stakes communication of events between processes.**

- If you are relying on signals to be the driver in your program, you should rethink your application design.

- There are many drawbacks to using POSIX/Real-Time signals for asynchronous communication.

- **The best way to handle interprocess communication is to use, well, interprocess communication methods specifically designed for your task at hand.**

Handling Signals(changing default behaviours)

There are strict limitations on the executable code inside a signal handler.

Most library and system calls are `async-signal-unsafe`, meaning they may not be used inside a signal handler **because they are not re-entrant***

```
int func(const char *str) {
  static char buffer[200];
  strncpy(buffer, str, 199);
  # Here is where we get paused
  printf("%s\n", buffer)
}
```

*Thus, a signal handler cannot, in general, call standard library functions.
Footnote 188 of §7.1.4 in ISO/IEC 9899:201x - Programming languages

Handling Signals

```c
int func(const char *str) {
  static char buffer[200];
  strncpy(buffer, str, 199);
  # Here is where we get paused
  printf("%s\n", buffer)
}
```

1. We execute `(func("Hello"))`
2. The string gets copied over to the buffer completely (strcmp(buffer, "Hello") == 0)
3. A signal is delivered and the function state freezes, we also stop accepting any new signals until after the handler (we do this for convenience)
4. We execute `func("World")`
5. Now (strcmp(buffer, "World") == 0) and the buffer is printed out "World".
6. We resume the interrupted function and now print out the buffer once again "World" instead of what the function call originally intended "Hello"

Guaranteeing that your functions are signal handler safe can't be solved by removing shared buffers.
Calling `printf`, which uses `malloc` as part of the signal handler, is unsafe and will result in **undefined behavior**.

A safe way to avoid this behavior is to set a variable and let the program resume operating.
The design pattern also helps us in designing programs that can receive signals twice and operate correctly.

```c
int pleaseStop ; // See notes on why "volatile sig_atomic_t" is better

void handle_sigint(int signal) {
  pleaseStop = 1;
}

int main() {
  signal(SIGINT, handle_sigint);
  pleaseStop = 0;
  while (!pleaseStop) {
    /* application logic here */
  }
  /* clean up code here */
}
```

Here, we need to provide a hint to the compiler and the CPU core that will execute the `main()` loop. **We need to prevent compiler optimization.**

```c
volatile sig_atomic_t pleaseStop;
```

Sigaction (changes actions for a specific signal)

```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

sa_handler specifies the action to be associated with *signum*:
The actions are:

```
struct sigaction {
  void (*sa_handler)(int);
  void (*sa_sigaction)(int, siginfo_t *, void *);
  sigset_t sa_mask;
  int sa_flags;
};
```

* **SIG_DFL** for the default action.

* **SIG_IGN** to ignore this signal.

* A pointer to a signal handling function. This function receives the signal number as its only argument.

From man signal:q
WARNING: the behavior of signal() varies across UNIX versions, and has also varied historically across different versions of Linux. Avoid its use: use sigaction(2) instead.

```
signal(SIGALRM, myhandler);
```

The equivalent `sigaction` code is:

```
struct sigaction sa;
sa.sa_handler = myhandler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

sigaction(SIGALRM, &sa, NULL)
```

we typically may also set the mask and the flags field.

The mask is a temporary signal mask used during the signal handler execution.

If the thread serving the signal is interrupted in the middle of a system call, the `SA_RESTART` flag will automatically restart some system calls that otherwise would have returned early with EINTR error.

The latter means we can simplify the rest of code somewhat because a restart loop may no longer be required.

```
sigfillset(&sa.sa_mask);
sa.sa_flags = SA_RESTART; /* Restart system calls if interrupted by handler */
```

Blocking Signals

- A signal may be blocked, which means that it will not be delivered until it is later unblocked.
- Each thread in a process has an independent signal mask, which indicates the set of signals that the thread is currently blocking.
- A thread can manipulate its signal mask using **`pthread_sigmask()`.**
- In a single-threaded application, **`sigprocmask()`** can be used to manipulate the signal mask.
- The behavior of **sigprocmask** in a multi-threaded process is “unspecified”!
- A child created via fork inherits a copy of its parent's signal mask; the signal mask is preserved across **`execve`**.

Functions for manipulation of POSIX signal sets

sigset_t data type is used to represent a signal set. Returns 0 on success,

- *int* **sigemptyset** *(sigset_t *set);*
  - initializes to empty (all of the defined signals are excluded).
- *int* **sigfillset** *(sigset_t *set);*
  - initializes to full (include all of the defined signals.)
  - `sigfillset(&set); // all signals`

- *int* **sigaddset** *(sigset_t *set, int signum);*
  - adds the signal signum to the signal set set.
  - All sigaddset does is modify set; it does not block or unblock any signals.
  - `sigaddset(&set, SIGINT); // should first initialize with sig empty of fillset!`
- *int* **sigdelset** *(sigset_t *set, int signum);*
  - removes the signal signum from the signal set set.

- *int* **sigismember** *(const sigset_t *set, int signum)*
  - tests what signals are in a signal set:
  - Returns 1 if it is a member.

```
int sigprocmask (int how,
                 const sigset_t *restrict set,
                 sigset_t *restrict oldset);
```

The **how** argument determines how the signal mask is changed, and must be one of the following values:

- **SIG_BLOCK**
  - Block the signals in set—add them to the existing mask. In other words, the new mask is the union of the existing mask and set.
- **SIG_UNBLOCK**
  - Unblock the signals in set—remove them from the existing mask.
- **SIG_SETMASK**
  - Use set for the mask; ignore the previous value of the mask.

You can’t block the **SIGKILL** and **SIGSTOP** signals, but if the signal set includes these, sigprocmask just ignores them instead of returning an error status.

```
sigset_t set, oldset;
sigaddset(&set, SIGINT); // Ooops!
sigprocmask(SIG_SETMASK, &set, &oldset)
```

Correct code initializes the set to be all on or all off. For example,

```
sigfillset(&set); // all signals
sigprocmask(SIG_SETMASK, &set, NULL); // Block all the signals which can be blocked

sigemptyset(&set); // no signals
sigprocmask(SIG_SETMASK, &set, NULL); // set the mask to be empty again
```

```c
#include <stdio.h> #include <stdlib.h>
#include <signal.h> #include <unistd.h>
int main(void){
   sigset_t set, oldset, pendingset;
   sigemptyset(&set);
   sigaddset(&set, SIGINT);

   sigprocmask(SIG_BLOCK, &set, &oldset);
   printf("Old set was %8.8ld.\n", oldset);

   sigpending(&pendingset);
   printf("Pending set is %8.8ld.\n", pendingset);

   kill(getpid(), SIGINT);

   sigpending(&pendingset);
   printf("Pending set is %8.8ld.\n", pendingset);
   sigprocmask(SIG_UNBLOCK, &set, &oldset);
   /* The program terminates with a SIGINT */
   return (EXIT_SUCCESS);
}
```

Signals in Child Processes and Threads

Blocking signals is similar in multi-threaded programs to single-threaded programs with the following translation.

1. Use `pthread_sigmask` instead of `sigprocmask`
2. Block a signal in all threads to prevent its asynchronous delivery

```
pthread_sigmask(...); // set my mask to block delivery of some signals
pthread_create(...); // new thread will start with a copy of the same mask
```

The easiest method to ensure a signal is blocked in all threads is to set the signal mask in the main thread before new threads are created.

```
sigemptyset(&set);
sigaddset(&set, SIGQUIT);
sigaddset(&set, SIGINT);
pthread_sigmask(SIG_BLOCK, &set, NULL);

// this thread and the new thread will block SIGQUIT and SIGINT
pthread_create(&thread_id, NULL, myfunc, funcparam);
```

Just as we saw with `sigprocmask`, `pthread_sigmask` includes a ‘how’ parameter that defines how the signal set is to be used:

```
pthread_sigmask(SIG_SETMASK, &set, NULL) - replace the thread's mask with given signal set
```

```
pthread_sigmask(SIG_BLOCK, &set, NULL) - add the signal set to the thread's mask
```

```
pthread_sigmask(SIG_UNBLOCK, &set, NULL) - remove the signal set from the thread's mask
```

Even though programs from the outside can't send signals to specific threads, you can do that internally with `pthread_kill(pthread_t thread, int sig)`. In the example below, the newly created thread executing `func` will be interrupted by `SIGINT`

```
pthread_create(&tid, NULL, func, args);
```

```
pthread_kill(tid, SIGINT);
```

```
pthread_kill(pthread_self(), SIGKILL); // send SIGKILL to myself
```

As a word of warning `pthread_kill(threadid, SIGKILL)` will kill the entire process. Though individual threads can set a signal mask, the signal disposition is *per-process* not *per-thread*. This means `sigaction` can be called from any thread because you will be setting a signal handler for all threads in the process.